

AI for Economic Research: Dynamic Models, Language, and Agents

Lecture 1.3b: The AI/RL Alternative

Zhigang Feng

Spring 2026

Topic 1.3b Agenda

1. **The Robot-Taxi Problem** — RL framing of dynamic programming
2. **Monte Carlo, Function Approximation, Sampled Data** — the AI toolkit
3. **HA Models as Middle Ground** — bridging two traditions
4. **Numerical Techniques: Classical vs. AI Frontier**
5. **Course Roadmap and Programming Environment**

The AI/RL Alternative

AI/Reinforcement Learning: The Robot-Taxi Problem

- **Example:** Training an autonomous vehicle to navigate a city
- **State Space:** Camera images, LiDAR, GPS, speed, ...
 - Single camera frame: $1920 \times 1080 \times 3 \approx 6$ million dimensions
 - Continuous, high-dimensional, partially observable
- **Action Space:** Steering, acceleration, braking (continuous)
- **Environment:** Transition dynamics $P(s'|s, a)$
 - Physics, other drivers, pedestrians, weather
 - *Unknown and impossible to fully specify*
 - No “First Welfare Theorem” to simplify
- **Contrast with Macro:** No closed-form model; must learn from experience

RL Formulation: The Same Bellman Equation

- **Markov Decision Process (MDP):** (S, A, P, r, β)
- **Bellman Equation for Optimal Value:**

$$v^*(s) = \max_a \left\{ r(s, a) + \beta \int v^*(s') P(ds'|s, a) \right\}$$

- **Bellman Equation for Q-Function:**

$$Q^*(s, a) = r(s, a) + \beta \int \max_{a'} Q^*(s', a') P(ds'|s, a)$$

- **Key Insight:** Theoretical foundation is *identical* to economics.
 - Bellman equation, contraction mapping, fixed point—all apply
- **The difference is computational:** How do we solve this when P is unknown and S is enormous?

Three Fundamental Challenges

Challenge	Classical DP	Modern AI/RL
Dimensionality	Low ($d \leq 5$)	Very high ($d \geq 10^6$)
Environment	Fully known	Unknown, learned
Data	Regular grids	Random samples

- **Dimensionality:** Cannot discretize; must use function approximation
- **Unknown Environment:** Cannot evaluate $\mathbb{E}[v(s')]$ analytically; estimate from samples (Monte Carlo)
- **Irregular Data:** Samples from trajectories, not grids; must generalize

Monte Carlo: Turning Samples into Expectations

- **The Problem:** Computing $\mathbb{E}[v(s')|s, a] = \int v(s')P(ds'|s, a)$
 - Classical DP: Enumerate all s' , weight by known $P(s'|s, a)$
 - High dimensions or unknown P : Infeasible
- **Monte Carlo Estimation:**

$$\mathbb{E}[v(s')] \approx \frac{1}{N} \sum_{i=1}^N v(s'_i), \quad s'_i \sim P(\cdot|s, a)$$

- Generate samples by *simulating* or *interacting* with environment
 - Law of Large Numbers: Converges as $N \rightarrow \infty$
- **Key Advantage:** Cost scales with N , *not* dimension
 - Works equally well in 2 or 2 million dimensions
 - This is how RL escapes the curse of dimensionality

Function Approximation: Learning from Irregular Data

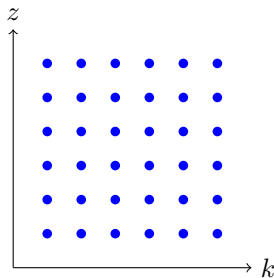
- **Classical DP:** Value function as table $\{v(s_i)\}_{i=1}^N$
 - Regular grid, pre-specified, covers entire state space
 - Interpolation straightforward

- **RL with Function Approximation:**

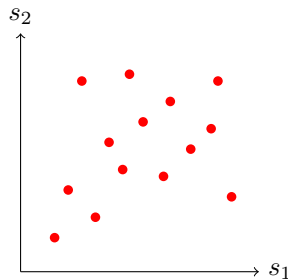
$$v(s) \approx v_\theta(s) = \text{NeuralNetwork}_\theta(s)$$

- Parameters θ learned from sampled data
 - Data from trajectories: $(s_0, a_0, r_0, s_1, a_1, r_1, \dots)$
 - Points concentrate where policy visits (not uniform)
- **Challenges:**
 - Must generalize to unseen states
 - Approximation errors can compound (instability)
 - No convergence guarantee (unlike tabular DP)

From Grids to Samples: A Paradigm Shift



Classical DP
Regular Grid



Modern RL
Sampled Trajectories

- **Left:** Compute $v(s)$ at every grid point; interpolate
- **Right:** Observe (s, a, r, s') tuples; fit $v_\theta(s)$ to minimize prediction error

Heterogeneous Agent Models: Between Two Worlds

- **Individual Problem:** Low-dimensional (a, z) —classical methods work
- **Aggregate State:** Distribution μ —infinite-dimensional
- **Traditional Approaches:**
 - Krusell-Smith: Approximate μ by moments $(\bar{K}, \sigma_K, \dots)$
 - Bounded rationality: Simple forecasting rules
 - Iterate until forecasts approximately consistent
- **Modern Approaches (Research Frontier):**
 - Neural networks to represent policy: $g_\theta(a, z, \mu)$
 - Monte Carlo simulation of agent distribution
 - Learn equilibrium conditions end-to-end
- **Key Insight:** Heterogeneous agent macro is a middle ground—we *know* economic structure but face RL-like computational challenges.

Overview: Two Traditions, Shared Foundations

	Quantitative Macro	AI / Reinforcement Learning
Theory	Bellman equation	Bellman equation
Foundation	Contraction mapping	Contraction mapping
Environment	Known (model-based)	Unknown (learn from data)
State Space	Low-dimensional	Very high-dimensional
Computation	Grid + interpolation	Samples + neural nets
Strength	Exploits structure	Scales to complexity
Weakness	Curse of dimensionality	Sample inefficiency

The Opportunity: Combine economic structure with ML scalability

- Use theory to reduce dimensionality where possible
- Use neural networks where analytical solutions fail
- Use Monte Carlo to handle high-dimensional distributions

Numerical Techniques: Classical Methods

- **Function Approximation and Interpolation**
 - Linear/cubic splines, Chebyshev polynomials
 - Approximating policy and value functions
- **Numerical Optimization**
 - Newton-Raphson, gradient descent, grid search
 - Finite differences, automatic differentiation (PyTorch)
- **Stochastic Process Approximation**
 - Tauchen, Rouwenhorst methods for discretizing AR(1)
- **Perturbation and Projection**
 - Linearization around steady state
 - Higher-order perturbation (2nd/3rd order for risk)
 - Projection methods with basis functions

Numerical Techniques: The AI Frontier

- **Deep Neural Networks**
 - Universal approximators for high-dimensional functions
 - Policy networks $\pi_{\theta}(s)$, value networks $v_{\theta}(s)$
- **Monte Carlo and Simulation**
 - Estimating expectations without discretization
 - Variance reduction techniques
- **Stochastic Optimization**
 - SGD, Adam, and variants
 - Training on mini-batches of sampled data
- **Reinforcement Learning**
 - Q-learning, policy gradient, actor-critic
 - Solving DP when environment is unknown or complex
- **Adaptive Methods:** Sparse grids, active learning for efficient sampling

What This Course Will Explore

- **Classical Foundations:**

- Dynamic programming theory (existence, uniqueness, convergence)
- Numerical methods: VFI, PFI, Euler iteration, projection, perturbation

- **Machine Learning Tools:**

- Function approximation with neural networks
- Monte Carlo methods and variance reduction
- Stochastic optimization (SGD, Adam)

- **Bridging the Gap:**

- Deep learning for solving Bellman equations
- Neural network solutions to heterogeneous agent models
- Exploiting economic structure in ML architectures

- **Goal:** Equip you to work at the frontier where economic theory meets computational innovation.

Programming Environment

- **Python**

- High-level, readable, vast ecosystem (NumPy, SciPy, Pandas)
- Excellent for prototyping; vectorized libraries for speed

- **PyTorch**

- GPU-accelerated tensor computing
- Dynamic computational graphs with automatic differentiation
- Essential for modern high-dimensional solving

- **Quick Start:**

```
import torch
import torch.nn as nn

v_net = nn.Sequential(
    nn.Linear(2, 64), nn.Tanh(),
    nn.Linear(64, 64), nn.Tanh(),
    nn.Linear(64, 1)
)
```

Summary: The Big Picture

1. **Theory:** Bellman equation + contraction mapping = foundation for both economics and AI
2. **Classical Macro:** Exploits known structure; limited by dimensionality
3. **Competitive Equilibrium:** Defined by optimality + market clearing; simplified by representative agent
4. **Heterogeneous Agents:** Distribution as state breaks classical methods
5. **Calibration & Estimation:** Tie the model to data; SMM and Bayesian methods
6. **Accuracy:** Always check Euler equation residuals; $|EEE| < 10^{-4}$ is the target
7. **Modern AI/RL:** Scales via Monte Carlo + neural networks; sample-intensive
8. **The Opportunity:** Combine economic structure with ML scalability

This course: Learn to work at the intersection.